AEROSPACE SOFTWARE ARCHITECTURE

ORBITALX BACKEND

Full Architectural Directory Specification & Mentor Defense Strategy Dossier

Mission Operations & Telemetry Control System

OrbitalX is an enterprise-grade satellite ground segment and mission control backend developed with Node.js and Express 5. Designed around a decoupled 4-Tier Layered Architecture, it orchestrates continuous Keplerian orbital dynamics, real-time subsystem telemetry, secure CubeSat telecommands, and aerospace flight-rule AI anomaly diagnostics with resilient dual-engine database failover.

Node.js 20+ | Express 5.2 | MongoDB Atlas / In-Memory Fallback | Keplerian Orbital Engine | JWT & RBAC Security | Aerospace AI Diagnostics | CubeSat 1U-16U Pipeline

PROJECT
OrbitalX Space Operations

ARCHITECTURE PATTERN
4-Tier Layered MVC / Service

TARGET AUDIENCE
Technical Mentors & Evaluators

ENVIRONMENT
Production & Zero-Config Dev

COMPLIANCE & SECURITY
Helmet, RBAC, Audit Logging

DOCUMENT VERSION
v2.4.0 (Enterprise Release)

1 Architectural Philosophy & High-Level System Design

The OrbitalX backend is structured around a **Separation of Concerns (SoC)** paradigm, implementing a battle-tested **4-Tier Layered MVC & Domain Service Pattern**. This ensures that network transport mechanisms, business rules, data persistence, and mathematical simulation loops remain strictly decoupled.

1. Transport & Middleware Tier `server.js / middleware`

Handles incoming HTTP ingress, TLS termination headers via **Helmet**, Gzip/Brotli payload compression, CORS security enforcement, and token-based authentication (`requireAuth` & `requireRole`).

2. Routing & Controller Tier `src/routes & controllers`

Validates request parameters and delegates execution. Controllers remain clean and declarative by wrapping all asynchronous operations inside a centralized `asyncHandler` and returning standardized `ApiResponse` envelopes.

3. Domain & Simulation Services `src/services`

Independent, persistent execution engines that handle continuous Keplerian orbit calculations, stochastic telemetry noise, anomaly generation, and aerospace flight-rule diagnostics independent of HTTP request lifecycles.

4. Persistence & Data Models `src/models & config`

Mongoose Object Data Modeling (ODM) with strongly typed schemas, compound indexing, and a resilient dual-engine database layer supporting cloud MongoDB Atlas with seamless zero-dependency In-Memory fallback.

Key Architectural Decision for Mentors

Why not put telemetry calculations inside the controller?
Satellites in orbit continuously generate telemetry regardless of whether an operator is viewing a dashboard. By decoupling the `telemetrySimulator.js` service from the HTTP request cycle, telemetry is continuously updated and indexed into MongoDB at 10-second intervals, allowing the API to serve sub-millisecond cached and queried orbital states to thousands of concurrent clients.

2 Complete Backend Directory Tree & Component Mapping

The following filesystem tree details the exact physical layout of the `OrbitalX/backend` repository:

```
backend/
├── src/
│   ├── config/
│   │   └── db.js                    # Dual-Engine MongoDB (Atlas SRV + In-Memory Fallback)
│   ├── controllers/
│   │   ├── adminController.js       # System health, metrics & node-level management
│   │   ├── aiController.js          # Aerospace Flight-Rule AI anomaly triage & recovery
│   │   ├── alertController.js       # Critical threshold trigger processing & resolution
│   │   ├── analyticsController.js   # Fleet-wide aggregate telemetry & health scoring
│   │   ├── auditController.js       # Immutable compliance & security event queries
│   │   ├── authController.js        # Operator login, registration & JWT issuance
│   │   ├── communicationController.js  # Uplink/Downlink pass scheduling & RF status
│   │   ├── cubesatCommandController.js # 1U-16U CubeSat telecommands & state mutations
│   │   ├── groundStationController.js  # Ground antenna tracking coordinates & coverage
│   │   ├── missionController.js     # Spacecraft mission phase & objective management
│   │   ├── notificationController.js   # Real-time operator push notifications
│   │   ├── satelliteController.js   # Satellite asset registry & orbital parameters
│   │   ├── telemetryController.js   # Time-series telemetry querying & real-time push
│   │   └── userController.js        # User profile management & RBAC role assignments
│   ├── middleware/
│   │   ├── auth.js                  # JWT Bearer verification & Role-Based Access Control (RBAC)
│   │   └── errorHandler.js          # Centralized unhandled error & ApiError interceptor
│   ├── models/
│   │   ├── Alert.js                 # Spacecraft threshold alerts & anomaly schema
│   │   ├── AuditLog.js              # Immutable user actions, telecommand timestamps & IPs
│   │   ├── Command.js               # Telecommand states (SENT, PROCESSING, ACKNOWLEDGED)
│   │   ├── Communication.js         # Ground pass communications & signal budgets
│   │   ├── GroundStation.js         # Earth antenna locations, azimuth/elevation limits
│   │   ├── Mission.js               # Active space mission metadata & milestone tracking
│   │   ├── Notification.js          # Operator system notifications and priority flags
│   │   ├── Satellite.js             # Satellite hardware profile, NORAD ID & orbit model
│   │   ├── Telemetry.js             # High-frequency time-series telemetry metrics
│   │   └── User.js                  # Operator accounts, bcrypt password hashes & roles
│   ├── routes/                      # 13 REST API endpoints corresponding to controllers
│   ├── services/
│   │   ├── telemetrySimulator.js    # Physics-based orbital motion & anomaly generator
│   │   └── auditService.js          # Asynchronous audit log persistence helper
│   ├── utils/
│   │   ├── ApiError.js              # Operational error class with HTTP status codes
│   │   ├── ApiResponse.js           # Standardized JSON response envelope
│   │   ├── asyncHandler.js          # Promise wrapper eliminating try-catch boilerplate
│   │   ├── cubeSatInitialStates.js  # Baseline bus, power, and thermal configurations
│   │   ├── password.js              # Bcrypt hashing and validation utilities
│   │   ├── seedData.js              # Automatic initialization of satellites, stations & admin
│   │   └── token.js                 # JWT token signing and verification helper
│   └── scripts/
│       └── clearData.js             # Database wiping & sanitization maintenance script
├── tests/
│   └── securityAudit.js           # Automated penetration test & authentication security suite
├── server.js                        # Application entry point, HTTP server bootstrap & DB wiring
├── package.json                     # Dependency manifest, scripts, and runtime configuration
├── Dockerfile                       # Multi-stage production container image specification
├── docker-compose.yml               # Container orchestration with MongoDB & networks
├── .env / .env.example              # Environment configuration (Port, JWT secret, MongoDB URI)
└── .gitignore / .dockerignore       # Build exclusion and repository hygiene rules
```

3 Deep-Dive: 4 High-Value Engineering Feats

1. Dual-Engine Resilient Database Architecture (`src/config/db.js`)

One of the most frequent friction points in aerospace and distributed software evaluation is database unavailability. OrbitalX implements a **zero-failure database strategy**:

- **DNS SRV Resolution Hardening:** Configures Google (`8.8.8.8`) and Cloudflare (`1.1.1.1`) upstream DNS servers to prevent Windows network stack resolution failures on Atlas connection strings.
- **Automatic Dynamic In-Memory Failover:** If the cloud MongoDB Atlas URI fails to connect within a 5-second timeout, the system seamlessly spins up a local `MongoMemoryServer` instance without terminating the server.
- **Buffer Command Safety:** `mongoose.set('bufferCommands', false)` prevents silent thread-blocking when the database is temporarily unreachable.
- **Resource Hygiene:** Automatically purges orphaned `mongo-mem-*` directories in the OS temporary directory on startup and teardown to prevent disk exhaustion.

2. Physics-Based Keplerian Orbital Telemetry Engine (`src/services/telemetrySimulator.js`)

Instead of broadcasting static mock data, OrbitalX runs a high-fidelity continuous orbital mechanics loop:

```
// Continuous Keplerian orbital motion along fixed orbital plane
const nowSec = Date.now() / 1000;
const incRad = (sat.inclination || 51.6) * (Math.PI / 180);
const periodSec = isLEO ? 5700 : isMEO ? 43200 : 86400; // 95 min for LEO
const meanMotion = (2 * Math.PI) / periodSec;
const u = (nowSec * meanMotion + phaseOffset) % (2 * Math.PI); // Orbital position
const sinLat = Math.sin(incRad) * Math.sin(u);
const simLat = (Math.asin(Math.max(-1, Math.min(1, sinLat))) * (180 / Math.PI));
```

- Accurately calculates satellite Latitude and Longitude by accounting for **Earth's 24-hour sidereal rotation**.
- Simulates real orbital altitude classes: **LEO (550 km, 7.6 km/s)**, **MEO (20,200 km, 3.88 km/s)**, and **GEO (35,786 km, 3.07 km/s)**.
- Features a stochastic 5% anomaly injection module that simulates realistic space environmental degradation (solar array shading, battery brownout, and thermal spikes), instantly broadcasting alerts.

3. Autonomous Aerospace Flight-Rules AI Diagnostics (`src/controllers/aiController.js`)

Integrates an aerospace flight-rule knowledge base covering six critical satellite subsystems:

- **EPS (Electrical Power):** Load shedding, bus switching, and battery trickle commands.
- **ADCS (Attitude Control):** B-dot magnetic detumble damping, wheel despin, and safe sun pointing.
- **COMMS (Communications & RF):** Power amplifier throttling to 250mW, backup antenna switching, and beacon recovery.
- **TCS (Thermal Control):** Attitude thermal rolls and survival heater override circuits.
- **OBC (On-Board Computer):** MRAM golden partition reboots and hung I2C bus clearing pulses.

4. CubeSat 1U-16U Telecommand & Audit Lifecycle (`src/controllers/cubesatCommandController.js`)

Commands follow a strict finite-state machine (`SENT` → `PROCESSING` → `ACKNOWLEDGED` / `FAILED`). Every single command execution writes an immutable record to `AuditLog.js` capturing the Operator User ID, satellite ID, client IP

address, and payload parameters.

4 End-to-End Request-Response Lifecycle Trace

To demonstrate architectural mastery to your mentor, trace a high-security action such as dispatching a telecommand to a CubeSat: `POST /api/telemetry/cubesat/command`.

Stage	Layer / File	Action Performed
1. Ingress & TLS	`server.js`	**Helmet** injects CSP, X-Frame-Options, and HSTS headers. **Compression** activates Gzip/Brotli. Body parser extracts JSON with a 10MB safety limit.
2. Authentication	`src/middleware/auth.js` (`requireAuth`)	Extracts the `Authorization: Bearer <token>` header. Verifies cryptographic signature via `jwt.verify()`. Queries `User.findById()` and attaches the sanitized operator object to `req.user`.
3. Authorization	`src/middleware/auth.js` (`requireRole`)	Inspects `req.user.role` against allowed roles (`['OPERATOR', 'ADMIN']`). If a viewer tries to issue a command, returns an immediate `403 Forbidden` via `ApiError`.
4. Route Dispatch	`src/routes/telemetryRoutes.js`	Matches endpoint pattern and invokes `cubesatCommandController.sendCubeSatCommand` wrapped in `asyncHandler`.
5. Business Logic	`src/controllers/cubesatCommandController.js`	Validates target satellite exists. Increments the sequential telecommand counter (`commandSequenceCounter++`). Mutates the in-memory or persisted subsystem state (e.g. switches attitude mode or toggles solar arrays).
6. Audit Trail	`src/models/AuditLog.js` & `Notification.js`	Asynchronously writes an immutable audit log entry containing the operator ID, target satellite, command name, and client IP address. Dispatches an in-app notification to the operations room.
7. Standard Envelope	`src/utils/ApiResponse.js`	Constructs an RFC-compliant response object: `{ statusCode: 200, data: {...}, message: "Telecommand executed successfully", success: true }`.
8. Fault Interception	`src/middleware/errorHandler.js`	If any promise rejects at any point, execution immediately jumps to the centralized error handler, preventing server crashes and returning safe JSON without exposing stack traces in production.

5 How to Explain It to Your Mentor (Step-by-Step Script)

When presenting this backend to your mentor, your goal is to demonstrate **architectural maturity**, **defensive engineering**, and **domain-specific knowledge**. Use the scripts below verbatim.

> **STEP 1: THE 60-SECOND EXECUTIVE PITCH (START WITH THIS)**
>
> *"Good morning/afternoon, Sir/Ma'am. OrbitalX is a mission control backend built for satellite ground operations and telemetry telemetry processing. Instead of building a simple CRUD application, I designed this backend using an enterprise 4-Tier Layered MVC and Service Architecture.*
>
> *It decouples network transport from business logic, features an autonomous Keplerian orbital mechanics simulation engine that runs in the background, implements aerospace flight-rule AI diagnostics for satellite subsystem failures, and guarantees zero-downtime development through an automated dual-engine database failover between MongoDB Atlas and an In-Memory MongoDB engine. Every privileged command is strictly enforced through JWT Role-Based Access Control and tracked in an immutable audit trail."*

> **STEP 2: EXPLAINING THE FOLDER STRUCTURE (OPEN VS CODE TREE)**
>
> *"Let me walk you through our folder hierarchy, which follows strict Separation of Concerns:*
>
> *• **src/config/db.js**: Connects to MongoDB Atlas, but if external DNS or network fails, it automatically instantiates an In-Memory MongoDB server on the fly so the platform never fails to boot.*
> *• **src/routes/ & controllers/**: We maintain 13 dedicated domains—such as Satellites, Telemetry, CubeSat Commands, and AI. Every controller method is wrapped in a custom `asyncHandler`, eliminating redundant try-catch blocks.*
> *• **src/services/telemetrySimulator.js**: This is our background physics engine. Every 10 seconds, it calculates true Keplerian orbital positions, Earth sidereal rotation, and subsystem temperatures, while injecting realistic anomalies.*
> *• **src/middleware/**: Provides defense-in-depth with Helmet headers, Gzip compression, and role-based guards (`requireRole`).*
> *• **src/models/**: 10 strongly typed Mongoose schemas indexed for high-frequency telemetry lookups."*

> **STEP 3: THE 3 TECHNICAL INNOVATIONS TO HIGHLIGHT**
>
> *"Three specific engineering highlights distinguish this codebase from standard web backends:*
>
> *1. **Resilience Engineering:** We solved the classic 'MongoDB Atlas connection timeout' problem by implementing dual-mode connectivity with Google and Cloudflare DNS fallbacks.*
> *2. **Autonomous AI Diagnostics:** In `aiController.js`, we encoded real aerospace flight rules for EPS, ADCS, and COMMS subsystems so the system can autonomously recommend step-by-step telecommands to recover a tumbling or overheated satellite.*
> *3. **Defensive Telecommand Pipeline:** CubeSat commands are stateful, sequence-counted, and immutably logged into `AuditLog` with operator identity and IP address for compliance."*

6 Mentor Defense: 8 Tough Questions & Verbatim Answers

Q1 Why did you choose Express 5 and Node.js instead of NestJS or Python/FastAPI?

"Express 5 provides native asynchronous error handling without the framework overhead or heavy reflection metadata of NestJS. Node.js's event loop and non-blocking I/O make it exceptionally suited for high-frequency concurrent telemetry data streams and asynchronous I/O. For heavy numerical orbital propagation, we kept the mathematical calculations lightweight and event-loop friendly, with clean service boundaries so that high-compute orbital ephemeris can be offloaded to a dedicated Python or WebAssembly microservice if required."

Q2 Why is the telemetry simulation in a background service instead of generated on the frontend?

"In real aerospace operations, satellites broadcast telemetry to ground stations regardless of whether any human operator is logged into a dashboard. If the simulation ran on the client browser, two operators viewing the same satellite would see conflicting coordinates and desynchronized telemetry. By anchoring the telemetry simulator in `src/services/telemetrySimulator.js`, our backend acts as the single source of truth (SSoT), persisting time-series telemetry records in MongoDB that any client can query."

Q3 How do you ensure data consistency and prevent server crashes when errors happen?

"We implement a three-layer defense: First, our custom `asyncHandler` wraps every asynchronous controller route and forwards uncaught exceptions directly to Express's `next(err)`. Second, our centralized `errorHandler.js` formats errors into a predictable JSON envelope while hiding internal stack traces in production. Third, critical operational errors are codified using our custom `ApiError` class with explicit HTTP status codes."

Q4 What is your security posture against unauthorized telecommands?

"We practice **Defense-in-Depth**: 1. **Network layer:** Helmet enforces HTTP security headers, and strict CORS configuration allows only whitelisted frontend origins. 2. **Identity layer:** Stateless JWT tokens signed with HMAC-SHA256 require verification on every request. 3. **Access Control:** Granular RBAC (`requireRole('OPERATOR', 'ADMIN')`) prevents standard Viewers from issuing telecommands. 4. **Accountability:** Every command execution immediately writes an immutable entry into `AuditLog` with the operator's User ID, timestamp, and client IP address."

Q5 How would this architecture scale if you deployed 10,000 satellites?

"Because the Express application tier is completely stateless, we can horizontally scale the Node.js API processes behind an Nginx or AWS ALB reverse proxy. For the high-frequency telemetry ingest, we would introduce a distributed message queue like Apache Kafka or Redis Streams between the telemetry receiver and MongoDB, and partition our MongoDB telemetry collection using time-series collections or sharding on `{ satelliteId: 1, timestamp: -1 }`, which is already indexed in our schema."

Q6 Why do you have both Controllers and Services? What is the architectural distinction?

"**Controllers** are responsible for HTTP transport concerns—parsing query parameters, reading request headers, and sending the formatted HTTP response. **Services** encapsulate pure business logic and operational processes that are protocol-agnostic. For instance, `telemetrySimulator.js` runs on an internal timer loop without any knowledge of HTTP requests, and `auditService.js` handles compliance logging that can be invoked by controllers, cron tasks, or external queues."

7 Mentor Presentation Checklist & Key Metrics

Demonstration Checklist

Live Demo

- Show terminal booting with `npm run dev`
- Point out the DB connection log (Atlas or In-Memory fallback)
- Show the automated `seedData()` population
- Demonstrate `telemetrySimulator` logging 10s cycles
- Open `src/config/db.js` to show resilience design
- Open `src/controllers/aiController.js` to show flight rules
- Open `src/middleware/auth.js` to show RBAC enforcement

Key Metrics to Quote

Architecture

- **Architecture:** 4-Tier Layered MVC + Domain Service
- **Route Domains:** 13 dedicated API endpoints
- **Controllers:** 14 business controllers
- **Models:** 10 indexed Mongoose schemas
- **Physics Engine:** Keplerian LEO/MEO/GEO calculations
- **Security:** JWT, Helmet, bcrypt, Audit Trail
- **Database Strategy:** Dual-Engine failover in <5s

Golden Advice for Mentor Evaluation

Always steer the conversation toward **resilience** and **separation of concerns**. Mentors respect candidates who understand *why* a certain design pattern was chosen, how it handles edge cases (like network drops or unauthorized commands), and how it scales to production.